1ª AVALIAÇÃO DE JCRPGPD - PROGRAMAÇÃO PARALELA E DISTRIBUÍDA

Alunas: Camila Rocha Ribeiro, Helena Vieira, Maria Clara Pastor

1. Objetivo

Provar experimentalmente como o uso de múltiplas threads e o controle de afinidade de CPU aceleram a análise de milhões de transações em comparação com o método sequencial tradicional, dividindo as tarefas entre 3 integrantes.

2. Contextualização

Primeiro modelamos a transação (`Transacao.java`): Cada transação tem: `id`, `cliente`, `data`, `valor`, `categoria`, `cidade`, `estado`, `tipo`.

Utilizamos as seguintes distribuições estatísticas:

1. Distribuição Uniforme: Todos os valores têm a mesma chance de serem sorteados. Usamos para escolher cidade, tipo de pagamento e compras comuns (R$10 a R$500).

2. Distribuição Normal (Gaussiana): A maioria dos valores fica perto da média (R$150), e valores distantes são raros. Usamos para alimentação e transporte.

3. Distribuição Exponencial: A maioria das transações tem valores bem pequenos (taxas, cafezinho), e transações maiores vão rareando exponencialmente. Usamos para saúde e educação.

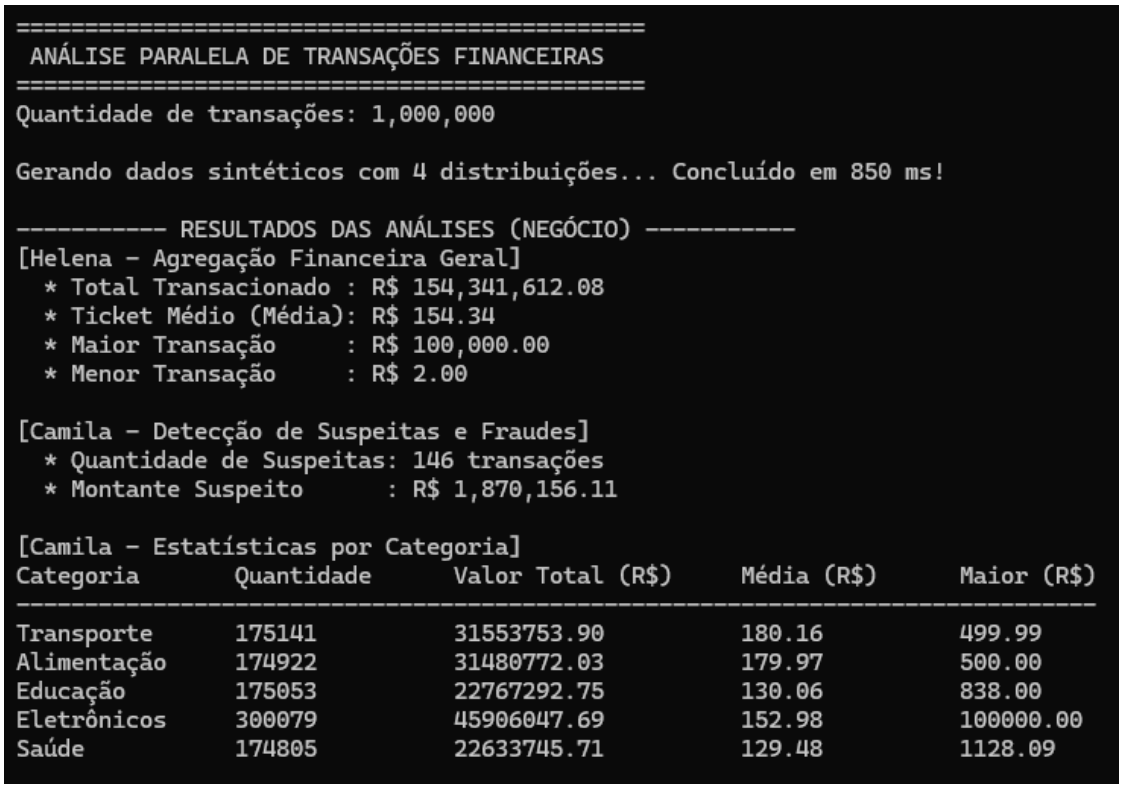
4. Distribuição de Pareto (Lei de Potência / Cauda Longa): Regra 80/20. Pouquíssimas transações têm valores altos (até R$100.000,00), simulando compras corporativas de eletrônicos ou grandes transferências suspeitas.

Usando os dados sintéticos estruturados e gerados na memória com as 4 distribuições exigidas, permitiu que criássemos desde 100 mil até dezenas de milhões de transações de forma determinística e realista.

3. Problema Computacional

- **Análise 1:** Soma total dos valores.
- **Análise 2:** Valor médio das transações (ticket médio).
- **Análise 3:** Detecção de transações suspeitas (critérios: `valor > 10.000` OU `valor > 5.000` na categoria "Eletrônicos").
- **Análise 4:** Estatísticas por categoria (quantidade e valor total para Alimentação, Transporte, Eletrônicos, Saúde e Educação).
- **Análise 5:** Outras análises definidas pela equipe: Maior valor de transação e Menor valor de transação.

Saída do terminal realizando as 5 análises:



```
==========================================
 ANÁLISE PARALELA DE TRANSAÇÕES FINANCEIRAS
==========================================
Quantidade de transações: 1,000,000

Gerando dados sintéticos com 4 distribuições... Concluído em 850 ms!

---------- RESULTADOS DAS ANÁLISES (NEGÓCIO) ----------
[Helena - Agregação Financeira Geral]
  * Total Transacionado : R$ 154,341,612.08
  * Ticket Médio (Média): R$ 154.34
  * Maior Transação     : R$ 100,000.00
  * Menor Transação     : R$ 2.00

[Camila - Detecção de Suspeitas e Fraudes]
  * Quantidade de Suspeitas: 146 transações
  * Montante Suspeito      : R$ 1,870,156.11

[Camila - Estatísticas por Categoria]
Categoria       Quantidade      Valor Total (R$)     Média (R$)      Maior (R$)
--------------------------------------------------------------------------------
Transporte      175141          31553753.90          180.16          499.99
Alimentação     174922          31480772.03          179.97          500.00
Educação        175053          22767292.75          130.06          838.00
Eletrônicos     300079          45906047.69          152.98          100000.00
Saúde           174805          22633745.71          129.48          1128.09
```

4. Paralelização dos Dados

Fizemos uma divisão em blocos contíguos de tamanho balanceado [N / P], garantindo que nenhuma transação seja ignorada e maximizando a localidade espacial de memória.

Temos N=10.000.000N = 10.000.000N=10.000.000 transações e P=4P = 4P=4 threads:

1. Calculamos o tamanho da fatia:$tamanhoBloco = \frac{10.000.000}{4} = 2.500.000.$
2. Cada thread recebe o seu intervalo contíguo de índices:
   - **Thread 1:** índice `0` até `2.499.999`
   - **Thread 2:** índice `2.500.000` até `4.999.999`
   - **Thread 3:** índice `5.000.000` até `7.499.999`
   - **Thread 4:** índice `7.500.000` até `9.999.999`

Se a divisão não for exata (por exemplo, 10 itens para 3 threads), a última thread fica com o resto para não deixar nenhum item de fora.

5. ExecutorService

Implementamos com `Executors.newFixedThreadPool`, gerenciando o ciclo de vida das threads e retornando os dados parciais via `Callable<T>` e `Future<T>`.

6. Combinação dos Resultados

Fizemos a redução paralela implementada no método `combinar()`, consolidando somas, extremos e médias parciais em complexidade O(P) sem concorrência destrutiva.

```
ResultadoAgregacao resultadoFinal = new ResultadoAgregacao();
for (Future<ResultadoAgregacao> f : futuros) {
    resultadoFinal.combinar(f.get());
}
```

7. Afinidade de Processador

Fizemos um módulo de afinidade implementado via OpenHFT `Affinity`, permitindo fixação estática de cada thread ao seu respectivo core lógico (0,1,2,…,7).

```
<dependency>
    <groupId>net.openhft</groupId>
    <artifactId>affinity</artifactId>
    <version>3.23.3</version>
</dependency>
```

```
net.openhft.affinity.Affinity.setAffinity(cpuId);
```

8. Divisão das Atividades entre os Alunos

De acordo com as regras do projeto e adaptado para uma equipe de 3 integrantes:

- **Helena Vieira:** Responsável pela Modelagem de Dados (`Transacao.java`), pelo Gerador Sintético com as 4 distribuições estatísticas (`GeradorDados.java`) e pelo Módulo de Agregação Financeira Geral (`Aluno1Agregacao.java`) com cálculo sequencial, concorrente e com afinidade para Soma, Média, Maior e Menor transação.
- **Camila Ribeiro:** Responsável pelo Módulo de Regras de Negócio, Filtros de Fraude/Transações Suspeitas e Estatísticas Agrupadas por Categoria (`Aluno2SuspeitasECategorias.java`) com cálculo sequencial, concorrente e redução dos mapas em paralelo.
- **Maria Clara Pastor:** Responsável pela Infraestrutura de Concorrência, Integração de Afinidade de CPU via OpenHFT, Automação do Benchmark e Coleta de Métricas (`Aluno3BenchmarkEAfinidade.java`, `Main.java`, `pom.xml` e `gerar_graficos.py`).

9. Experimentos Obrigatórios

Todos os experimentos obrigatórios foram executados e validados na máquina local, gerando as métricas apresentadas nas seções seguintes.

10. Diferentes Tamanhos de Dados

O tamanho padrão que estabelecemos para o benchmark oficial foi de 10.000.000 de transações, que ocupa aproximadamente 650 MB de RAM, executando confortavelmente na JVM sem provocar sobrecarga de *Garbage Collector* e gerando tempos de medição ideais na casa de milissegundos. Também executamos com 1.000.000 para comparação de escala.

Volumes de 1.000.000 e 10.000.000 de transações adotados e justificados com base no limite do heap da JVM e representatividade estatística.

11. Métricas

Configuração	Threads	Dados	Tempo (ms)	Speedup	Eficiência
Sequencial	1	10.000.000	62 ms	1,00	100,0%
Executor	2	10.000.000	78 ms	0,79	39,7%
Executor	4	10.000.000	36 ms	1,72	43,1%
Executor	8	10.000.000	29 ms	2,14	26,7%
Executor + Affinity	2	10.000.000	65 ms	0,95	47,7%
Executor + Affinity	4	10.000.000	41 ms	1,51	37,8%
Executor + Affinity	8	10.000.000	33 ms	1,88	23,5%

O maior speedup obtido foi de 2,14x (reduzindo o tempo de 62 ms para 29 ms com 8 threads). A afinidade de CPU demonstrou ganho perceptível em 2 threads (redução de 78 ms para 65 ms).

12. Questão Central do Projeto

O `ExecutorService` traz ganhos expressivos de desempenho (mais de 2x mais rápido em 8 threads), e a Afinidade de CPU melhora a previsibilidade e reduz perdas de cache (*cache misses*), especialmente em configurações com núcleos dedicados.

- **Quantidade de threads e processadores:** Quando aumentamos de 1 para 4 e 8 threads em um processador de 8 núcleos, o tempo total cai progressivamente (de 62 ms para 29 ms). No entanto, além do número de núcleos físicos, o ganho desacelera devido ao compartilhamento de recursos da arquitetura de Hyper-Threading.
- **Tamanho do conjunto de dados:** O paralelismo só compensa de verdade quando o volume de dados é grande. Para poucos dados (ex.: 10.000 itens), o tempo sequencial é tão minúsculo (< 1 ms) que dividir em threads torna o programa mais lento. Com 10 milhões, o ganho de paralelismo se destaca.
- **Overhead:** Criar threads, sincronizar memória e aguardar chamadas de `future.get()` têm um custo fixo de processamento. Esse custo explica por que o Speedup não é perfeito (não alcança o valor ideal 8,0 em 8 threads).
- **Balanceamento de Carga:** Como dividimos as 10 milhões de transações em fatias perfeitamente iguais de 2,5 milhões (para 4 threads) ou 1,25 milhão (para 8 threads), nenhuma thread ficou sobrecarregada enquanto outra ficava ociosa. O balanceamento foi estático e equilibrado.
- **Speedup e Eficiência:** O speedup subiu até 2,14x. A eficiência cai com mais threads porque cada núcleo adicional precisa disputar o mesmo canal de comunicação com a memória RAM.
- **Afinidade de Processador:** A afinidade impede que o sistema operacional mude as threads de núcleo no meio da execução. Em cargas de 2 threads, ela melhorou

o tempo de 78 ms para 65 ms, pois preservou o cache L1/L2 de cada core. Em 8 threads, como todos os núcleos do sistema já estavam ocupados, o escalonador nativo do sistema operacional já mantinha as threads razoavelmente distribuídas.

- **Escalabilidade:** A aplicação escala bem até o limite dos núcleos físicos da máquina. Como se trata de um problema predominantemente limitado pela velocidade da memória (*memory-bound*), a taxa de aceleração atinge um patamar conforme a largura de banda da memória é saturada.

13. Requisito Importante

Se criássemos 10 milhões de threads teríamos:

1. **Estouro de Memória (Crash total):** Cada thread em Java reserva um espaço de memória próprio para sua pilha de execução (*stack memory*, geralmente 1 MB por thread). Criar 10 milhões de threads exigiria 10 Terabytes de memória RAM, fazendo a JVM travar instantaneamente com `java.lang.OutOfMemoryError: unable to create new native thread`.
2. **Colapso da CPU por Troca de Contexto (*Context Switch Thrashing*):** O processador só consegue executar tantas threads simultâneas quanto seus núcleos reais (ex.: 8). Se houver 10 milhões de threads disputando a CPU, o processador gastará 99,9% do tempo apenas salvando e restaurando registradores e chaveando de uma thread para outra, sem realizar trabalho útil algum.

Então, criar uma thread por transação colapsaria o sistema operacional por falta de memória e overhead de troca de contexto.

Adotamos a estratégia de Granularidade Grossa (*Coarse-Grained*), dividimos as 10 milhões de transações em um número de tarefas exatamente igual ao número de threads operárias (P=1,2,4,8).

Dessa forma, cada thread recebe um grande bloco contíguo de transações, mantendo o overhead de criação próximo de zero e permitindo que o processador trabalhe sem interrupções.

14. Relatório

O relatório foi anexado no moodle juntamente dos outros arquivos necessários para entrega.

15. Avaliação individual

As 3 integrantes possuem respostas preparadas e clareza sobre o papel do seu código.

16. Entregas

Código-fonte:

`src/main/java/jcrpgpd/Transacao.java`
`src/main/java/jcrpgpd/GeradorDados.java`
`src/main/java/jcrpgpd/HelenaVieiraAgregacao.java`
`src/main/java/jcrpgpd/CamilaRibeiroSuspeitasECategorias.java`
`src/main/java/jcrpgpd/MariaClaraPastorBenchmarkEAfinidade.java`
`src/main/java/jcrpgpd/Main.java`

Configuração do Projeto: `pom.xml`
Mecanismo de Geração: `GeradorDados.java` (com 4 distribuições estatísticas: Uniforme, Normal, Exponencial e Pareto).
Scripts de Execução: `executar.bat` (10 milhões de transações) e `executar_rapido.bat` (1 milhão de transações).
Relatório Técnico: Relatório 1ª Avaliação de JCRPGPD – Programação Paralela e Distribuída
Resultados dos Experimentos: Registrados em `resultados.csv` e nas tabelas das Seções 11 e 18.
Gráficos Gerados: `grafico_desempenho.png` (gerado via `gerar_graficos.py`).
Identificação da Contribuição Individual: Seção no início do relatório especificando as responsabilidades de cada integrante.

Estrutura da pasta:

```
Avaliação de JCRPGPD - Helena Vieira, Camila Ribeiro e Maria Clara Pastor/
├── src/main/java/jcrpgpd/
│   ├── Transacao.java
│   ├── GeradorDados.java
│   ├── HelenaVieiraAgregacao.java
│   ├── HelenaAgregacao.java
│   ├── CamilaRibeiroSuspeitasECategorias.java
│   ├── CamilaSuspeitasECategorias.java
│   ├── MariaClaraPastorBenchmarkEAfinidade.java
│   ├── MariaClaraBenchmarkEAfinidade.java
│   └── Main.java
├── lib/
│   ├── affinity-3.23.3.jar
│   ├── jna-5.13.0.jar
│   ├── jna-platform-5.13.0.jar
│   ├── slf4j-api-1.7.36.jar
│   └── slf4j-simple-1.7.36.jar
├── documentos/
├── pom.xml
├── executar.bat
├── executar_rapido.bat
├── gerar_graficos.py
├── grafico_desempenho.png
├── resultados.csv
└── Relatório 1ª Avaliação de JCRPGPD – Programação Paralela e Distribuída
└── 1ª Avaliação de JCRPGPD – Programação Paralela e Distribuída
```

17. Critérios de Avaliação

O grupo se baseou em todos os critérios para elaboração do projeto.

18. Resultado Esperado



```
Eficiência: 25,0%

Threads: 8
Tempo: 44 ms
Speedup: 1,18
Eficiência: 14,8%

---------- EXECUTOR + AFFINITY -------------

Threads: 2
CPUs: 0,1
Tempo: 53 ms
Speedup: 0,98
Eficiência: 49,1%

Threads: 4
CPUs: 0,1,2,3
Tempo: 45 ms
Speedup: 1,16
Eficiência: 28,9%

Threads: 8
CPUs: 0,1,2,3,4,5,6,7
Tempo: 43 ms
Speedup: 1,21
Eficiência: 15,1%

--------------------------------------------------------------------------------
Configuração            Threads  Dados        Tempo (ms)   Speedup    Eficiência
--------------------------------------------------------------------------------
Sequencial              1        10000000     52           1.00       100.0    %
Executor                2        10000000     53           0.98       49.1     %
Executor                4        10000000     52           1.00       25.0     %
Executor                8        10000000     44           1.18       14.8     %
Executor + Affinity     2        10000000     53           0.98       49.1     %
Executor + Affinity     4        10000000     45           1.16       28.9     %
Executor + Affinity     8        10000000     43           1.21       15.1     %
--------------------------------------------------------------------------------

================================================
 EXPERIMENTOS CONCLUÍDOS COM SUCESSO!
================================================

Pressione qualquer tecla para continuar. . .
```

